# LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation

**Abstract.**

Graph Convolution Network (GCN) has become new state-of-the-art for collaborative filtering. Nevertheless, the reasons of its effectiveness for recommendation are not well understood. Existing work that adapts GCN to recommendation lacks thorough ablation analyses on GCN, which is originally designed for graph classification tasks and equipped with many neural network operations. However, we empirically find that the two most common designs in GCNs — feature transformation and nonlinear activation — contribute little to the performance of collaborative filtering. Even worse, including them adds to the difficulty of training and degrades recommendation performance.

In this work, we aim to simplify the design of GCN to make it more concise and appropriate for recommendation. We propose a new model named LightGCN, including only the most essential component in GCN — neighborhood aggregation — for collaborative filtering. Specifically, LightGCN learns user and item embeddings by linearly propagating them on the user-item interaction graph, and uses the weighted sum of the embeddings learned at all layers as the final embedding. Such simple, linear, and neat model is much easier to implement and train, exhibiting substantial improvements (about 16.0% relative improvement on average) over Neural Graph Collaborative Filtering (NGCF) — a state-of-the-art GCN-based recommender model — under exactly the same experimental setting. Further analyses are provided towards the rationality of the simple LightGCN from both analytical and empirical perspectives. Our implementations are available in both TensorFlow[11]https://github.com/kuandeng/LightGCN and PyTorch[22]https://github.com/gusye1234/pytorch-light-gcn.

## 1.Introduction

To alleviate information overload on the web, recommender system has been widely deployed to perform personalized information filtering (Ying et al., 2018; Covington et al., 2016; Yuan et al., 2020). The core of recommender system is to predict whether a user will interact with an item, e.g., click, rate, purchase, among other forms of interactions. As such, collaborative filtering (CF), which focuses on exploiting the past user-item interactions to achieve the prediction, remains to be a fundamental task towards effective personalized recommendation (He et al., 2017; Liang et al., 2018; Wang et al., 2019b; Ebesu et al., 2018).

The most common paradigm for CF is to learn latent features (a.k.a. embedding) to represent a user and an item, and perform prediction based on the embedding

vectors (He et al., 2017; Cheng et al., 2018). Matrix factorization is an early such model, which directly projects the single ID of a user to her embedding (Koren et al., 2009). Later on, several research find that augmenting user ID with the her interaction history as the input can improve the quality of embedding. For example, SVD++ (Koren, 2008) demonstrates the benefits of user interaction history in predicting user numerical ratings, and Neural Attentive Item Similarity (NAIS) (He et al., 2018) differentiates the importance of items in the interaction history and shows improvements in predicting item ranking. In view of user-item interaction graph, these improvements can be seen as coming from using the subgraph structure of a user — more specifically, her one-hop neighbors — to improve the embedding learning.

To deepen the use of subgraph structure with high-hop neighbors, Wang et al. (Wang et al., 2019b) recently proposes NGCF and achieves state-of-the-art performance for CF. It takes inspiration from the Graph Convolution Network (GCN) (Kipf and Welling, 2017; Hamilton et al., 2017), following the same propagation rule to refine embeddings: feature transformation, neighborhood aggregation, and nonlinear activation. Although NGCF has shown promising results, we argue that its designs are rather heavy and burdensome — many operations are directly inherited from GCN without justification. As a result, they are not necessarily useful for the CF task. To be specific, GCN is originally proposed for node classification on attributed graph, where each node has rich attributes as input features; whereas in user-item interaction graph for CF, each node (user or item) is only described by a one-hot ID, which has no concrete semantics besides being an identifier. In such a case, given the ID embedding as the input, performing multiple layers of nonlinear feature transformation — which is the key to the success of modern neural networks (He et al., 2016) — will bring no benefits, but negatively increases the difficulty for model training.

To validate our thoughts, we perform extensive ablation studies on NGCF. With rigorous controlled experiments (on the same data splits and evaluation protocol), we draw the conclusion that the two operations inherited from GCN — feature transformation and nonlinear activation — has no contribution on NGCF's effectiveness. Even more surprising, removing them leads to significant accuracy improvements. This reflects the issues of adding operations that are useless for the target task in graph neural network, which not only brings no benefits, but rather degrades model effectiveness. Motivated by these empirical findings, we present a new model named LightGCN, including the most essential component of GCN — neighborhood aggregation — for collaborative filtering. Specifically, after associating each user (item) with an ID embedding, we propagate the embeddings on the user-item interaction graph to refine them. We then combine the embeddings learned at different propagation layers with a weighted sum to obtain the final embedding for prediction. The whole model is simple and elegant, which not only is easier to train, but also achieves better empirical performance than NGCF and other state-of-the-art methods like Mult-VAE (Liang et al., 2018).

To summarize, this work makes the following main contributions:

We empirically show that two common designs in GCN, feature transformation and nonlinear activation, have no positive effect on the effectiveness of collaborative filtering.

We propose LightGCN, which largely simplifies the model design by including only the most essential components in GCN for recommendation.

We empirically compare LightGCN with NGCF by following the same setting and demonstrate substantial improvements. In-depth analyses are provided towards the rationality of LightGCN from both technical and empirical perspectives.

# 2.Preliminaries

We first introduce NGCF (Wang et al., 2019b), a representative and state-of-the-art GCN model for recommendation. We then perform ablation studies on NGCF to judge the usefulness of each operation in NGCF. The novel contribution of this section is to show that the two common designs in GCNs, feature transformation and nonlinear activation, have no positive effect on collaborative filtering.

## 2.1.NGCF Brief

In the initial step, each user and item is associated with an ID embedding. Let $\mathrm{e}_u^{(0)}$ denote the ID embedding of user $u$ and $\mathrm{e}_i^{(0)}$ denote the ID embedding of item $i$. Then NGCF leverages the user-item interaction graph to propagate embeddings as:

where $\mathrm{e}_u^{(k)}$ and $\mathrm{e}_i^{(k)}$ respectively denote the refined embedding of user $u$ and item $i$ after $k$ layers propagation, $\sigma$ is the nonlinear activation function, $\mathcal{N}_u$ denotes the set of items that are interacted by user $u$, $\mathcal{N}_i$ denotes the set of users that interact with item $i$, and $\mathrm{W}_1$ and $\mathrm{W}_2$ are trainable weight matrix to perform feature transformation in each layer. By propagating $L$ layers, NGCF obtains $L+1$ embeddings to describe a user $(\mathrm{e}_u^{(0)}, \mathrm{e}_u^{(1)}, ..., \mathrm{e}_u^{(L)})$ and an item $(\mathrm{e}_i^{(0)}, \mathrm{e}_i^{(1)}, ..., \mathrm{e}_i^{(L)})$. It then concatenates these $L+1$ embeddings to obtain the final user embedding and item embedding, using inner product to generate the prediction score.

NGCF largely follows the standard GCN (Kipf and Welling, 2017), including the use of nonlinear activation function $\sigma(\cdot)$ and feature transformation matrices $\mathrm{W}_1$ and $\mathrm{W}_2$. However, we argue that the two operations are not as useful for collaborative filtering. In semi-supervised node classification, each node has rich semantic features as input, such as the title and abstract words of a paper. Thus performing multiple layers of nonlinear transformation is beneficial to feature learning. Nevertheless, in collaborative filtering, each node of user-item interaction graph only has an ID as input which has no concrete semantics. In this case, performing multiple nonlinear transformations will not contribute to learn better features; even worse, it may add the difficulties to train well. In the next subsection, we provide empirical evidence on this argument.

## 2.2.Empirical Explorations on NGCF

We conduct ablation studies on NGCF to explore the effect of nonlinear activation and feature transformation. We use the codes released by the authors of NGCF[3][3]https://github.com/xiangwang1223/neural_graph_collaborative_filtering, running experiments on the same data splits and evaluation protocol to keep the comparison as fair as possible. Since the core of GCN is to refine embeddings by

propagation, we are more interested in the embedding quality under the same embedding size. Thus, we change the way of obtaining final embedding from concatenation (i.e., $\mathbf{e}_u^* = \mathbf{e}_u^{(0)} \| \cdots \| \mathbf{e}_u^{(L)}$) to sum (i.e., $\mathbf{e}_u^* = \mathbf{e}_u^{(0)} + \cdots + \mathbf{e}_u^{(L)}$). Note that this change has little effect on NGCF's performance, but makes the following ablation studies more indicative of the embedding quality refined by GCN.

We implement three simplified variants of NGCF:

NGCF-f, which removes the feature transformation matrices $\mathrm{W}_1$ and $\mathrm{W}_2$.

NGCF-n, which removes the non-linear activation function $\sigma$.

NGCF-fn, which removes both the feature transformation matrices and non-linear activation function.

For the three variants, we keep all hyper-parameters (e.g., learning rate, regularization coefficient, dropout ratio, etc.) same as the optimal settings of NGCF. We report the results of the 2-layer setting on the Gowalla and Amazon-Book datasets in Table 1. As can be seen, removing feature transformation (i.e., NGCF-f) leads to consistent improvements over NGCF on all three datasets. In contrast, removing nonlinear activation does not affect the accuracy that much. However, if we remove nonlinear activation on the basis of removing feature transformation (i.e., NGCF-fn), the performance is improved significantly. From these observations, we conclude the findings that:

(1) Adding feature transformation imposes negative effect on NGCF, since removing it in both models of NGCF and NGCF-n improves the performance significantly;

(2) Adding nonlinear activation affects slightly when feature transformation is included, but it imposes negative effect when feature transformation is disabled.

(3) As a whole, feature transformation and nonlinear activation impose rather negative effect on NGCF, since by removing them simultaneously, NGCF-fn demonstrates large improvements over NGCF (9.57% relative improvement on recall).

To gain more insights into the scores obtained in Table 1 and understand why NGCF deteriorates with the two operations, we plot the curves of model status recorded by training loss and testing recall in Figure 1. As can be seen, NGCF-fn achieves a much lower training loss than NGCF, NGCF-f, and NGCF-n along the whole training process. Aligning with the curves of testing recall, we find that such lower training loss successfully transfers to better recommendation accuracy. The comparison between NGCF and NGCF-f shows the similar trend, except that the improvement margin is smaller.

From these evidences, we can draw the conclusion that the deterioration of NGCF stems from the training difficulty, rather than overfitting. Theoretically speaking, NGCF has higher representation power than NGCF-f, since setting the weight matrix $\mathrm{W}_1$ and $\mathrm{W}_2$ to identity matrix I can fully recover the NGCF-f model. However, in practice, NGCF demonstrates higher training loss and worse generalization performance than NGCF-f. And the incorporation of nonlinear activation further aggravates the discrepancy between representation power and generalization

performance. To round out this section, we claim that when designing model for recommendation, it is important to perform rigorous ablation studies to be clear about the impact of each operation. Otherwise, including less useful operations will complicate the model unnecessarily, increase the training difficulty, and even degrade model effectiveness.

# 3.Method

The former section demonstrates that NGCF is a heavy and burdensome GCN model for collaborative filtering. Driven by these findings, we set the goal of developing a light yet effective model by including the most essential ingredients of GCN for recommendation. The advantages of being simple are several-fold — more interpretable, practically easy to train and maintain, technically easy to analyze the model behavior and revise it towards more effective directions, and so on.

In this section, we first present our designed Light Graph Convolution Network (LightGCN) model, as illustrated in Figure 2. We then provide an in-depth analysis of LightGCN to show the rationality behind its simple design. Lastly, we describe how to do model training for recommendation.

## 3.1.LightGCN

The basic idea of GCN is to learning representation for nodes by smoothing features over the graph (Kipf and Welling, 2017; Wu et al., 2019a). To achieve this, it performs graph convolution iteratively, i.e., aggregating the features of neighbors as the new representation of a target node. Such neighborhood aggregation can be abstracted as:

The AGG is an aggregation function — the core of graph convolution — that considers the $k$-th layer's representation of the target node and its neighbor nodes. Many work have specified the AGG, such as the weighted sum aggregator in GIN (Xu et al., 2018), LSTM aggregator in GraphSAGE (Hamilton et al., 2017), and bilinear interaction aggregator in BGNN (Zhu et al., 2020) etc. However, most of the work ties feature transformation or nonlinear activation with the AGG function. Although they perform well on node or graph classification tasks that have semantic input features, they could be burdensome for collaborative filtering (see preliminary results in Section 2.2).

### 3.1.1.Light Graph Convolution (LGC)

In LightGCN, we adopt the simple weighted sum aggregator and abandon the use of feature transformation and nonlinear activation. The graph convolution operation (a.k.a., propagation rule (Wang et al., 2019b)) in LightGCN is defined as:

The symmetric normalization term $\frac{1}{\sqrt{|\mathcal{N}_u|}\sqrt{|\mathcal{N}_i|}}$ follows the design of standard GCN (Kipf and Welling, 2017), which can avoid the scale of embeddings increasing with graph convolution operations; other choices can also be applied here, such as

the $L_1$ norm, while empirically we find this symmetric normalization has good performance (see experiment results in Section 4.4.2).

It is worth noting that in LGC, we aggregate only the connected neighbors and do not integrate the target node itself (i.e., self-connection). This is different from most existing graph convolution operations (Wang et al., 2019b; Kipf and Welling, 2017; Velickovic et al., 2018; Hamilton et al., 2017; Zhu et al., 2020) that typically aggregate extended neighbors and need to handle the self-connection specially. The layer combination operation, to be introduced in the next subsection, essentially captures the same effect as self-connections. Thus, there is no need in LGC to include self-connections.

### 3.1.2.Layer Combination and Model Prediction

In LightGCN, the only trainable model parameters are the embeddings at the 0-th layer, i.e., $\mathrm{e}_u^{(0)}$ for all users and $\mathrm{e}_i^{(0)}$ for all items. When they are given, the embeddings at higher layers can be computed via LGC defined in Equation (3). After $K$ layers LGC, we further combine the embeddings obtained at each layer to form the final representation of a user (an item):

where $\alpha_k \geq 0$ denotes the importance of the $k$-th layer embedding in constituting the final embedding. It can be treated as a hyper-parameter to be tuned manually, or as a model parameter (e.g., output of an attention network (Chen et al., 2017)) to be optimized automatically. In our experiments, we find that setting $\alpha_k$ uniformly as $1 / (K+1)$ leads to good performance in general. Thus we do not design special component to optimize $\alpha_k$, to avoid complicating LightGCN unnecessarily and to keep its simplicity. The reasons that we perform layer combination to get final representations are three-fold. (1) With the increasing of the number of layers, the embeddings will be over-smoothed (Li et al., 2018). Thus simply using the last layer is problematic. (2) The embeddings at different layers capture different semantics. E.g., the first layer enforces smoothness on users and items that have interactions, the second layer smooths users (items) that have overlap on interacted items (users), and higher-layers capture higher-order proximity (Wang et al., 2019b). Thus combining them will make the representation more comprehensive. (3) Combining embeddings at different layers with weighted sum captures the effect of graph convolution with self-connections, an important trick in GCNs (proof sees Section 3.2.1).

The model prediction is defined as the inner product of user and item final representations:

which is used as the ranking score for recommendation generation.

### 3.1.3.Matrix Form

We provide the matrix form of LightGCN to facilitate implementation and discussion with existing models. Let the user-item interaction matrix be $\mathrm{R} \in \mathbb{R}^{M \times N}$ where $M$ and $N$ denote the number of users and items, respectively, and each entry $R_{u\ i}$ is 1 if $u$ has interacted with item $i$ otherwise 0. We then obtain the adjacency matrix of the user-item graph as

Let the 0-th layer embedding matrix be $\mathrm{E}^{(0)} \in \mathbb{R}^{(M+N)\times T}$, where $T$ is the embedding size. Then we can obtain the matrix equivalent form of LGC as:

where D is a $(M+N)\times(M+N)$ diagonal matrix, in which each entry $D_{i\ i}$ denotes the number of nonzero entries in the $i$-th row vector of the adjacency matrix A (also named as degree matrix). Lastly, we get the final embedding matrix used for model prediction as:

where $\tilde{\mathrm{A}} = \mathrm{D}^{-\frac{1}{2}}\ \mathrm{AD}^{-\frac{1}{2}}$ is the symmetrically normalized matrix.

## 3.2.Model Analysis

We conduct model analysis to demonstrate the rationality behind the simple design of LightGCN. First we discuss the connection with the Simplified GCN (SGCN) (Wu et al., 2019a), which is a recent linear GCN model that integrates self-connection into graph convolution; this analysis shows that by doing layer combination, LightGCN subsumes the effect of self-connection thus there is no need for LightGCN to add self-connection in adjacency matrix. Then we discuss the relation with the Approximate Personalized Propagation of Neural Predictions (APPNP) (Klicpera et al., 2019), which is recent GCN variant that addresses oversmoothing by inspiring from Personalized PageRank (Haveliwala, 2002); this analysis shows the underlying equivalence between LightGCN and APPNP, thus our LightGCN enjoys the sames benefits in propagating long-range with controllable oversmoothing. Lastly we analyze the second-layer LGC to show how it smooths a user with her second-order neighbors, providing more insights into the working mechanism of LightGCN.

### 3.2.1.Relation with SGCN

In (Wu et al., 2019a), the authors argue the unnecessary complexity of GCN for node classfication and propose SGCN, which simplifies GCN by removing nonlinearities and collapsing the weight matrices to one weight matrix. The graph convolution in SGCN is defined as[44]The weight matrix in SGCN can be absorbed into the 0-th layer embedding parameters, thus it is omitted in the analysis.:

where $\mathrm{I} \in \mathbb{R}^{(M+N)\times(M+N)}$ is an identity matrix, which is added on A to include self-connections. In the following analysis, we omit the $(\mathrm{D}+\mathrm{I})^{-\frac{1}{2}}$ terms for simplicity, since they only re-scale embeddings. In SGCN, the embeddings obtained at the last layer are used for downstream prediction task, which can be expressed as:

The above derivation shows that, inserting self-connection into A and propagating embeddings on it, is essentially equivalent to a weighted sum of the embeddings propagated at each LGC layer.

### 3.2.2.Relation with APPNP

In a recent work (Klicpera et al., 2019), the authors connect GCN with Personalized PageRank (Haveliwala, 2002), inspiring from which they propose a GCN variant

named APPNP that can propagate long range without the risk of oversmoothing. Inspired by the teleport design in Personalized PageRank, APPNP complements each propagation layer with the starting features (i.e., the 0-th layer embeddings), which can balance the need of preserving locality (i.e., staying close to the root node to alleviate oversmoothing) and leveraging the information from a large neighborhood. The propagation layer in APPNP is defined as:

where $\beta$ is the teleport probability to control the retaining of starting features in the propagation, and $\tilde{\mathrm{A}}$ denotes the normalized adjacency matrix. In APPNP, the last layer is used for final prediction, i.e.,

Aligning with Equation (8), we can see that by setting $\alpha_k$ accordingly, LightGCN can fully recover the prediction embedding used by APPNP. As such, LightGCN shares the strength of APPNP in combating oversmoothing — by setting the $\alpha$ properly, we allow using a large $K$ for long-range modeling with controllable oversmoothing.

Another minor difference is that APPNP adds self-connection into the adjacency matrix. However, as we have shown before, this is redundant due to the weighted sum of different layers.

#### 3.2.3.Second-Order Embedding Smoothness

Owing to the linearity and simplicity of LightGCN, we can draw more insights into how does it smooth embeddings. Here we analyze a 2-layer LightGCN to demonstrate its rationality. Taking the user side as an example, intuitively, the second layer smooths users that have overlap on the interacted items. More concretely, we have:

We can see that, if another user $v$ has co-interacted with the target user $u$, the smoothness strength of $v$ on $u$ is measured by the coefficient (otherwise 0):

This coefficient is rather interpretable: the influence of a second-order neighbor $v$ on $u$ is determined by 1) the number of co-interacted items, the more the larger; 2) the popularity of the co-interacted items, the less popularity (i.e., more indicative of user personalized preference) the larger; and 3) the activity of $v$, the less active the larger. Such interpretability well caters for the assumption of CF in measuring user similarity (Chen et al., 2019b; Wang et al., 2006) and evidences the reasonability of LightGCN. Due to the symmetric formulation of LightGCN, we can get similar analysis on the item side.

### 3.3.Model Training

The trainable parameters of LightGCN are only the embeddings of the 0-th layer, i.e., $\Theta = \left\{\mathrm{E}^{(0)}\right\}$; in other words, the model complexity is same as the standard matrix factorization (MF). We employ the Bayesian Personalized Ranking (BPR) loss (Rendle et al., 2009), which is a pairwise loss that encourages the prediction of an observed entry to be higher than its unobserved counterparts:

where $\lambda$ controls the $L_2$ regularization strength. We employ the Adam (Kingma and Ba, 2015) optimizer and use it in a mini-batch manner. We are aware of other advanced negative sampling strategies which might improve the LightGCN training,

such as the hard negative sampling (Rendle and Freudenthaler, 2014) and adversarial sampling (Ding et al., 2019). We leave this extension in the future since it is not the focus of this work.

Note that we do not introduce dropout mechanisms, which are commonly used in GCNs and NGCF. The reason is that we do not have feature transformation weight matrices in LightGCN, thus enforcing $L_2$ regularization on the embedding layer is sufficient to prevent overfitting. This showcases LightGCN's advantages of being simple — it is easier to train and tune than NGCF which additionally requires to tune two dropout ratios (node dropout and message dropout) and normalize the embedding of each layer to unit length.

Moreover, it is technically viable to also learn the layer combination coefficients $\{\alpha_k\}_{k=0}^{K}$, or parameterize them with an attention network. However, we find that learning $\alpha$ on training data does not lead improvement. This is probably because the training data does not contain sufficient signal to learn good $\alpha$ that can generalize to unknown data. We have also tried to learn $\alpha$ from validation data, as inspired by (Chen et al., 2019a) that learns hyper-parameters on validation data. The performance is slightly improved (less than $1\%$). We leave the exploration of optimal settings of $\alpha$ (e.g., personalizing it for different users and items) as future work.

# 4.Experiments

We first describe experimental settings, and then conduct detailed comparison with NGCF (Wang et al., 2019b), the method that is most relevant with LightGCN but more complicated (Section 4.2). We next compare with other state-of-the-art methods in Section 4.3. To justify the designs in LightGCN and reveal the reasons of its effectiveness, we perform ablation studies and embedding analyses in Section 4.4. The hyper-parameter study is finally presented in Section 4.5.

## 4.1.Experimental Settings

To reduce the experiment workload and keep the comparison fair, we closely follow the settings of the NGCF work (Wang et al., 2019b). We request the experimented datasets (including train/test splits) from the authors, for which the statistics are shown in Table 2. The Gowalla and Amazon-Book are exactly the same as the NGCF paper used, so we directly use the results in the NGCF paper. The only exception is the Yelp2018 data, which is a revised version. According to the authors, the previous version did not filter out cold-start items in the testing set, and they shared us the revised version only. Thus we re-run NGCF on the Yelp2018 data. The evaluation metrics are recall@20 and ndcg@20 computed by the all-ranking protocol — all items that are not interacted by a user are the candidates.

*The scores of NGCF on Gowalla and Amazon-Book are directly copied from Table 3 of the NGCF paper (https://arxiv.org/abs/1905.08108)

### 4.1.1.Compared Methods

The main competing method is NGCF, which has shown to outperform several methods including GCN-based models GC-MC (van den Berg et al., 2018) and PinSage (Ying et al., 2018), neural network-based models NeuMF (He et al., 2017) and CMN (Ebesu et al., 2018), and factorization-based models MF (Rendle et al., 2009) and HOP-Rec (Yang et al., 2018). As the comparison is done on the same datasets under the same evaluation protocol, we do not further compare with these methods. In addition to NGCF, we further compare with two relevant and competitive CF methods:

Mult-VAE (Liang et al., 2018). This is an item-based CF method based on the variational autoencoder (VAE). It assumes the data is generated from a multinomial distribution and using variational inference for parameter estimation. We run the codes released by the authors[55]https://github.com/dawenl/vae_cf, tuning the dropout ratio in $[0, 0.2, 0.5]$, and the $\beta$ in $[0.2, 0.4, 0.6, 0.8]$. The model architecture is the suggested one in the paper: $600 \rightarrow 200 \rightarrow 600$.

GRMF (Rao et al., 2015). This method smooths matrix factorization by adding the graph Laplacian regularizer. For fair comparison on item recommendation, we change the rating prediction loss to BPR loss. The objective function of GRMF is:

where $\lambda_g$ is searched in the range of $[1\ \ e^{-5}, 1\ \ e^{-4}, ..., 1\ \ e^{-1}]$. Moreover, we compare with a variant that adds normalization to graph Laplacian: $\lambda_g \left\| \frac{\mathrm{e}_u}{\sqrt{|\mathcal{N}_u|}} - \frac{\mathrm{e}_i}{\sqrt{|\mathcal{N}_i|}} \right\|^2$, which is termed as GRMF-norm. Other hyper-parameter settings are same as LightGCN. The two GRMF methods benchmark the performance of smoothing embeddings via Laplacian regularizer, while our LightGCN achieves embedding smoothing in the predictive model.

### 4.1.2.Hyper-parameter Settings

Same as NGCF, the embedding size is fixed to 64 for all models and the embedding parameters are initialized with the Xavier method (Glorot and Bengio, 2010). We optimize LightGCN with Adam (Kingma and Ba, 2015) and use the default learning rate of 0.001 and default mini-batch size of 1024 (on Amazon-Book, we increase the mini-batch size to 2048 for speed). The $L_2$ regularization coefficient $\lambda$ is searched in the range of $\{1\ \ e^{-6}, 1\ \ e^{-5}, ..., 1\ \ e^{-2}\}$, and in most cases the optimal value is . The layer combination coefficient is uniformly set to where is the number of layers. We test in the range of 1 to 4, and satisfactory performance can be achieved when equals to 3. The early stopping and validation strategies are the same as NGCF. Typically, 1000 epochs are sufficient for LightGCN to converge. Our implementations are available in both TensorFlow and PyTorch.
■